

[GAME] PYGAME 100 LESSONS

Complete Beginner's Guide to Game Development

Learn Python Game Programming Step-by-Step
With Code Examples and Clear Instructions

Created: August 27, 2026

Level: Beginner / Novice

Duration: ~40-50 hours of learning

TABLE OF CONTENTS

PART 1: SETUP AND BASICS (Lessons 1-10)

Getting Python, Pygame, and your first program

PART 2: DRAWING SHAPES (Lessons 11-20)

Lines, circles, rectangles, and colors

PART 3: KEYBOARD INPUT (Lessons 21-30)

Making things move with the keyboard

PART 4: MOUSE INPUT (Lessons 31-40)

Click, drag, and detect mouse position

PART 5: SPRITES AND OBJECTS (Lessons 41-50)

Creating game objects and animations

PART 6: COLLISIONS (Lessons 51-60)

Detecting when objects touch

PART 7: SOUNDS AND MUSIC (Lessons 61-70)

Adding audio to your games

PART 8: SCORE AND TEXT (Lessons 71-80)

Displaying text, scores, and game info

PART 9: GAME MECHANICS (Lessons 81-90)

Timers, spawning, and game loops

PART 10: COMPLETE GAME (Lessons 91-100)

Build a full game from scratch

LESSON 1: What is Pygame?

Pygame is a Python library that helps you make games. It gives you tools to draw pictures, handle keyboard clicks, and make sounds. Think of it like LEGO for games!

LESSON 2: Install Python on Windows

Go to python.org, download Python 3.11 or newer, and run the installer. Make sure to check 'Add Python to PATH' during installation.

LESSON 3: Install Python on Mac

Download Python from python.org or use Homebrew: `brew install python3`. Then verify with: `python3 --version`

LESSON 4: Install Python on Linux

Use your package manager: `sudo apt install python3 python3-pip` (Ubuntu/Debian) or `sudo dnf install python3` (Fedora)

LESSON 5: Create Your First Folder

Make a new folder called 'pygame_games' on your desktop. This is where all your game code will live.

LESSON 6: Install Pygame Library

Open Command Prompt (Windows) or Terminal (Mac/Linux) and type: `pip install pygame`

LESSON 7: Your First Program

Create a file called `game.py` with code that prints 'Hello Pygame!'. Run it with: `python game.py`

```
HINT - Try It:  
print('Hello Pygame!')
```

LESSON 8: Understanding the Game Loop

A game loop runs your code over and over (60 times per second). This makes animations smooth and responsive.

LESSON 9: Creating a Game Window

Write code that opens a 500x500 pixel window. Windows stay open using `pygame.display.update()`

LESSON 10: Your First Pygame Project

Combine everything: create a window, run a game loop, and close it when you press the X button.

[COMPLETE] PART 1 FINISHED! You have learned key concepts. Practice before moving forward!

LESSON 11: Understanding Colors

Colors are made with 3 numbers (R, G, B) from 0-255. Red = (255,0,0), Green = (0,255,0), Blue = (0,0,255)

LESSON 12: Draw a Rectangle

Use `pygame.draw.rect()` to draw boxes. Rectangle needs: screen, color, and (x, y, width, height)

LESSON 13: Draw a Circle

Use `pygame.draw.circle()` with: screen, color, (center_x, center_y), radius

LESSON 14: Draw a Line

Use `pygame.draw.line()` with: screen, color, (start_x, start_y), (end_x, end_y), thickness

LESSON 15: Draw Polygons

Use `pygame.draw.polygon()` to draw triangles and other shapes with multiple points

HINT - Try It:

```
pygame.draw.polygon(screen, (255,0,0), [(100,50), (150,100), (50,100)])
```

LESSON 16: Mix Colors Together

Create custom colors by mixing R, G, B values. Try (255, 128, 0) for orange!

LESSON 17: Fill the Background

Use `screen.fill(color)` at the start of your game loop to clear the screen

LESSON 18: Draw Multiple Shapes

Draw many shapes in one loop. Order matters - shapes drawn last appear on top

LESSON 19: Create a Simple Scene

Draw a background, sun, ground, and tree to practice shape drawing

LESSON 20: Add Animation to Shapes

Move shapes by changing their position every frame. Increase x each loop to move right.

[COMPLETE] PART 2 FINISHED! You have learned key concepts. Practice before moving forward!

LESSON 21: Detect Key Presses

Use `pygame.key.get_pressed()` to check if any key is held down

LESSON 22: Move Left with Arrow Keys

Check if LEFT arrow is pressed, then decrease the object's x position

LESSON 23: Move Right with Arrow Keys

Check if RIGHT arrow is pressed, then increase the object's x position

LESSON 24: Move Up with Arrow Keys

Check if UP arrow is pressed, then decrease the object's y position

LESSON 25: Move Down with Arrow Keys

Check if DOWN arrow is pressed, then increase the object's y position

LESSON 26: Move in All 4 Directions

Combine all 4 arrow keys so your object moves smoothly in any direction

```
HINT - Try It:  
keys = pygame.key.get_pressed()  
if keys[pygame.K_LEFT]: x -= 5
```

LESSON 27: Use WASD Keys Instead

W=Up, A=Left, S=Down, D=Right. Check `pygame.K_w`, `pygame.K_a`, etc.

LESSON 28: Control Speed of Movement

Instead of moving 1 pixel, move 5 pixels. Create a variable called `speed = 5`

LESSON 29: Bounce Off Screen Edges

When $x < 0$, set $x = 0$. When $x > \text{screen_width}$, set $x = \text{screen_width} - \text{size}$

LESSON 30: Wrap Around Screen

When object leaves right side, appear on left side. Creates a cool effect!

[COMPLETE] PART 3 FINISHED! You have learned key concepts. Practice before moving forward!

LESSON 31: Get Mouse Position

Use `pygame.mouse.get_pos()` to get (x, y) of mouse. Store in a variable.

LESSON 32: Draw at Mouse Position

Draw a circle where your mouse is. The circle follows your cursor!

LESSON 33: Detect Mouse Clicks

Check `pygame.mouse.get_pressed()` to see which mouse buttons are clicked

LESSON 34: Left Click Detection

Check if `mouse[0]` is True to detect left click

LESSON 35: Right Click Detection

Check if `mouse[2]` is True to detect right click

LESSON 36: Create Clickable Areas

Draw rectangles and check if mouse is inside using `rect.collidepoint(mouse_pos)`

LESSON 37: Change Color on Click

When a shape is clicked, change its color. Create a variable to track state.

LESSON 38: Draw When Dragging

While mouse button is held, draw shapes at mouse position. Releases when button lets go.

LESSON 39: Create a Paint Program

Combine drawing and mouse - create a simple program where you paint by clicking

LESSON 40: Detect Mouse Entering Shapes

Check if mouse position is inside a rectangle or circle. Highlight when hovered.

```
HINT - Try It:  
mouse_pos = pygame.mouse.get_pos()  
if rect.collidepoint(mouse_pos):
```

[COMPLETE] PART 4 FINISHED! You have learned key concepts. Practice before moving forward!

LESSON 41: What are Sprites?

Sprites are images in your game. They're small pictures of characters, enemies, or objects.

LESSON 42: Load an Image File

Use `pygame.image.load('filename.png')` to load an image. Store it in a variable.

LESSON 43: Display an Image

Use `screen.blit(image, (x, y))` to draw the image on screen at position x, y

LESSON 44: Scale an Image

Use `pygame.transform.scale(image, (width, height))` to make images bigger or smaller

LESSON 45: Rotate an Image

Use `pygame.transform.rotate(image, angle)` to spin images. Angle 90 = 90 degrees

LESSON 46: Create a Player Class

Make a class called Player with x, y, image. This organizes your code better.

LESSON 47: Move a Player Sprite

Add movement code to your Player class. Update x and y in a method.

LESSON 48: Create Multiple Sprites

Make a list of sprites: `enemies = [Enemy(), Enemy(), Enemy()]`. Store many in one list.

LESSON 49: Draw All Sprites

Loop through your sprite list and draw each one. Use a for loop: `for sprite in list:`

LESSON 50: Update All Sprites

Call an `update()` method on each sprite each frame. This moves them and animates them.

[COMPLETE] PART 5 FINISHED! You have learned key concepts. Practice before moving forward!

LESSON 51: Collision Detection Basics

Collisions happen when two things touch. We use `rectangles.colliderect()` to check.

LESSON 52: Get Rectangle from Image

Every image has a `rect`. Use `image.get_rect()` to get a rectangle for collision checking.

LESSON 53: Detect Collisions with Rect

Use `rect1.colliderect(rect2)` - returns `True` if they overlap, `False` otherwise

LESSON 54: Check Player and Enemy Collision

In your game loop, check if `player.rect.colliderect(enemy.rect)`

LESSON 55: Remove Objects on Collision

When something is hit, remove it from the list: `enemies.remove(enemy)`

LESSON 56: Lose Health on Collision

Create a health variable. When hit, `health -= 1`. Game over when `health == 0`

LESSON 57: Bounce Off Objects

When collision happens, reverse direction: `velocity_x = -velocity_x`

LESSON 58: Collect Items

Create coins or items. When player touches them, add points and remove the item.

```
HINT - Try It:
for coin in coins:
    if player.rect.colliderect(coin.rect): score += 10
```

LESSON 59: Check Circle Collisions

Use math to check if circles touch. Distance between centers < sum of radii means collision.

LESSON 60: Collision Groups and Lists

Organize collisions: check all bullets vs all enemies in a loop

[COMPLETE] PART 6 FINISHED! You have learned key concepts. Practice before moving forward!

LESSON 61: Load a Sound Effect

Use `pygame.mixer.Sound('sound.wav')` to load a sound file

LESSON 62: Play a Sound Once

Call `sound.play()` to play a sound effect one time

LESSON 63: Load Background Music

Use `pygame.mixer.music.load('song.mp3')` to load music

LESSON 64: Play Music in a Loop

Use `pygame.mixer.music.play(-1)` - the -1 means loop forever

LESSON 65: Stop Music

Use `pygame.mixer.music.stop()` to stop the music

LESSON 66: Control Volume

Use `sound.set_volume(0.5)` to make sound 50% loud. 1.0 = full volume, 0.0 = silent

LESSON 67: Play Sound on Event

When something happens (collision, click), play a sound effect

LESSON 68: Multiple Sound Channels

Play many sounds at once using `pygame.mixer.get_channel()`

LESSON 69: Fade Out Music

Use `pygame.mixer.music.fadeout(milliseconds)` to fade music out slowly

LESSON 70: Fade In Music

Use `set_volume(0)` then gradually increase it each frame for fade-in effect

[COMPLETE] PART 7 FINISHED! You have learned key concepts. Practice before moving forward!

LESSON 71: Create a Font

Use `pygame.font.Font('arial.ttf', size)` to create text with a specific font and size

LESSON 72: Render Text

Use `font.render(text, True, color)` to convert text to an image you can draw

LESSON 73: Display Text on Screen

Use `screen.blit(text_image, (x, y))` to draw text just like an image

LESSON 74: Create a Score Variable

Make a variable `score = 0`. Increase it when player collects items.

LESSON 75: Display Score

Show the score on screen: `'Score: ' + str(score)`

```
HINT - Try It:  
font = pygame.font.Font(None, 36)  
text = font.render('Score: ' + str(score), True, (255,255,255))
```

LESSON 76: Create a Game Over Screen

Display text when the game ends. Fill screen with color, show message, wait.

LESSON 77: Display Lives Remaining

Show how many lives left: `'Lives: ' + str(lives)`

LESSON 78: Display Health Bar

Draw a rectangle that gets smaller as health decreases

LESSON 79: Display Frames Per Second

Calculate `FPS = 1 / delta_time`. Show it on screen for debugging.

LESSON 80: Show Tutorial Text

Display instructions at game start. Wait 3 seconds, then hide.

[COMPLETE] PART 8 FINISHED! You have learned key concepts. Practice before moving forward!

LESSON 81: Create a Timer

Make timer = 0, increase every frame: timer += 1. Reset when timer > target.

LESSON 82: Spawn Enemies Every 2 Seconds

Use timer to spawn: if timer > 120 (2 seconds): enemies.append(Enemy())

LESSON 83: Create a Level System

When score reaches certain points, increase level and make game harder

LESSON 84: Increase Difficulty

Make enemies faster, add more enemies, or reduce player speed as game progresses

LESSON 85: Create Power-ups

Add special items that give temporary advantages: faster speed, invincible, etc.

LESSON 86: Countdown Timer

Start with time_left = 60. Decrease each frame. Game ends when time_left == 0

LESSON 87: Waves of Enemies

Spawn enemies in groups. When wave is cleared, start next wave with harder enemies

LESSON 88: Game States

Create states: MENU, PLAYING, GAME_OVER. Change state based on events.

LESSON 89: Pause Menu

When P is pressed, pause the game. Draw menu, wait for button press to unpause.

LESSON 90: High Score Saving

When game ends, check if new_score > high_score. Save high score to a file.

[COMPLETE] PART 9 FINISHED! You have learned key concepts. Practice before moving forward!

LESSON 91: Plan Your Game

Decide: What does player do? What are enemies? How to win/lose? Write it down!

LESSON 92: Choose Game Type

Simple options: Dodge enemies, Collect items, Shoot enemies, Simple puzzle

LESSON 93: Create Main Game Loop

Setup pygame, create window, run game loop, handle quit

LESSON 94: Add Player Movement

Let player move with arrow keys or WASD. Keep player on screen.

LESSON 95: Add Enemies/Obstacles

Spawn enemies that move. Make them disappear at screen edges.

LESSON 96: Add Collision Logic

Check collisions between player and enemies. Decrease health or collect points.

LESSON 97: Add Score and Display

Create score variable, increase on actions, display on screen

LESSON 98: Add Game Over Condition

When health = 0 or time runs out, show game over screen

LESSON 99: Add Sound Effects

Add sounds for: movement, collision, score increase, game over

```
HINT - Try It:  
import pygame.mixer  
sound = pygame.mixer.Sound('jump.wav')  
sound.play()
```

LESSON 100: Finish Your Game!

Test everything, fix bugs, celebrate! Share your game with friends!

[COMPLETE] PART 10 FINISHED! You have learned key concepts. Practice before moving forward!

CONGRATULATIONS!

You have completed all 100 lessons! You now have the skills to create your own games with Pygame.

Next Steps:

1. Review lessons you found challenging
2. Create your own original game
3. Add new features to existing games
4. Join a game development community
5. Share your games with others!

Happy Game Making!